

TITLE AND DATE

Document name: Project#5 EEPROM Lab Report

Document reference: christopherWard.cmpe311.spring25.project#4

Date of publication: May 5th, 2026

LEAD ENGINEER: Christopher Ward, UMBC

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION: This document is the project document for Project#5 of the CMPE311 class. It contains customer, technical and testing requirements as well as the design and results of the validation testing. The Project utilizes EEPROM on the Arduino Uno to save data across power failures.

DESCRIPTION: This design is to create a circuit on an Arduino Uno R3 development platform that allows the user of the program to independently specify an LED number and its blink interval. The blinking of the LEDs must continue asynchronously with any input from the user. Included in this document are the customer requirements obtained from the customer, the high-level technical requirements derived from those customer requirements, the design, the testing scenarios and requirements, and the results of testing. Appended to this document is the code executed and a video of those tests that required video documentation. The design must also allow a fan to spin with various speeds using a button as input. The design must implement freertos. The design must additionally be able to save data across power failures.

RESULT SUMMARY: The project was a success, the embedded system design meeting all testing and high-level requirements.

REFERENCES AND GLOSSARY

<to eliminate any ambiguity, include any specific terms or entities, any unique words, any clarification, and any abbreviations>

REFERENCES:

- ProjectAsyncTasking_draft – The definition of the project this document addresses
- CMPE310 Project #5 – A similar project assigned in CMPE310 in Fall 2025
- Arduino UNO R3 Product Reference Manual SKU A000066, 12/03/2024
- Async Programming in Arduino: Unleashing the Power of Non-Blocking Code, Mahdi Valizadeh, medium.com, 4/8/2024

DEFINITIONS:

“The User” – The person operating (not programming) the embedded system

“The System” – The embedded system being operated by The User

“The Customer” – The person(s) paying for the embedded system being designed and built

“The Developer” – The person(s) designing and building the System

“The Evaluator” – The person(s) that determine whether or not The System satisfies The Customer-requirements.

“The Customer-requirements” – The requirements defined by The Customer as satisfying The Contract.

“The Requirements” – The System’s high-level technical requirements derived from The Customer-requirements.

“The Educational-constraints” – Requirements imposed by the instructor unrelated to the embedded system that allow The System to be evaluated.

“The Company” – The organization The Customer has contracted with to build The System.

“The Contract” – The business document that legally binds The Company to provide some service or product to The Customer.

“serial-monitor” – The serial port used by the Arduino IDE to communicate with The User.

“The Reference-platform” – The configuration of The System used by The Developer to test and validate The System. For this class, The System is the Arduino compatible ELEGOO Uno R3 development board.

ACRONYMS AND ABBREVIATIONS:

Arduino – an Italian open-source hardware and software company; also refers to a development board created by the company

arduino.h – header for a library of convenience functions specific to the Arduino development platform

AVR – A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology

ELEGOO – A Chinese company that develops and markets 3D printers and accessories

IDE – Integrated Development Environment

gcc – front end for the GNU Compiler Collection

Github – A widely used distributed SVC (Software Version Control) system

LED – Light Emitting Diode

REQUIREMENTS

<customer, high-level and testing requirements>

CONVENTIONS:

Must, shall or will – your design must satisfy the requirement

May – your design may satisfy the requirement but doesn't have to

Informative – the intent of the following description is to make the requirement more understandable

All customer requirements are started with "C.#".

All high-level requirements are started with "HL.#".

All testing/validation requirements are started with "T.#"

CUSTOMER REQUIREMENTS: *<list the customer requirements which I gave you>*

C1. The User must be able to set the blink rate of two different LEDs.

C2. The User must be able to update the blink rate of each of the LEDs independently.

C3. The LED must blink at the set rate until The User tells the LED to blink at a different rate.

C4. The System must run upon an Arduino Uno R3 compatible development board.

C5. The blink rate of an LED must be expressed in terms of milliseconds

C6. The device must have a motor-driven fan

C7. The RPM of the fan must be controllable via a button

C8. The fan must have 4 speeds: OFF, LOW, MEDIUM and HIGH

C9. Each hit of the button must cycle the speeds from OFF to LOW, MEDIUM, HIGH, MEDIUM, LOW, OFF

C10. LOW must be able to move air with minimal sound and minimum feeling of flow

C11. MEDIUM must be able to move air with moderate sound and a moderate feeling of air flow

C12. HIGH must be able to move air quickly with moderate sound and a distinct feeling of air flow

HIGH-LEVEL TECHNICAL REQUIREMENTS: *<list the derived high-level technical requirements>*

HL.1. The User through the IDE serial monitor must be able to set the blink rate of two different LEDs. *(From requirement C1)*

HL.1.1. The blink rate of each LED must be able to be set independent of the other LED. *(From requirement C2)*

HL.1.2. The setting of an LED blink rate must not interfere with the blinking of the LEDs until the new LED and blink rate are specified. *(From requirements C2 and C3)*

HL.2. The user through the IDE serial monitor must set the blink rate in terms of once every N milliseconds *(From requirement C5)*

HL.3. The System must run upon an Arduino Uno R3 compatible development board. *(From requirement C4)*

HL.7 The device must have a DC motor-driven fan with an operating voltage between 3V and 12V

HL.8 The fan must operate at a minimum between 1500 and 6500 RPM

HL.9 The fan must draw less than 4 Amps

HL.10 The fan must be driven by a PWM signal with a maximum voltage of 5VDC

HL.11 The fan must have a maximum dimension less than 4"x 4"x 3" (not including cage)

HL.12 The RPM of the motor must be controllable using a single-pole, momentary-on, normally-off push button.

HL.13 The operation of the button and motor must have no noticeable effect upon the operation of previously implemented I/O (e.g. serial monitor and LEDs)

HL.14 The fan must have 4 speeds: OFF, LOW, MEDIUM and HIGH

HL.15 Each hit of the button must cycle the speeds from OFF to LOW to MEDIUM to HIGH to MEDIUM to LOW to OFF

HL.16 LOW must be able to move air with a sound volume less than 35dB and airflow levels between 30-45 CFM (Cubic Feet per Minute)

HL.17 MEDIUM must be able to move air with a sound volume less than 40dB and airflow levels between 45-70 CFM (Cubic Feet per Minute)

HL.18 HIGH must be able to move air with a sound volume less than 50dB and airflow levels between 70-100 CFM (Cubic Feet per Minute)

HL.19 The system and application software will be built upon a RTOS (Real-time Operating System) – added by Kidd, 4/1/30.

HL.20 The Task Manager and Task Dispatcher will be built upon the RTOS's task management framework – added by Kidd, 4/1/30.

HL.21 A mechanism must exist whereby tasks can temporarily save sensitive data, even across power interruptions. – added by Kidd, 4/20/30

DESIGN:

<include schematics, flow diagrams, pseudo code, equipment needed, assumptions, software, data requirements, etc>

DESIGN PRE-REQUISITES: *<include any equipment or other things you are required to build your design upon such as the configuration of the development platform>*

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better

DEVELOPMENT PLATFORM: *<include any equipment, SW, etc necessary to reproduce your design environment>*

1. See DESIGN PRE-REQUISITES above

ANY ADDITIONAL DESIGN CONSIDERATIONS: *<include anything else needed for someone to understand and reproduce your design>*

1. None

See Appendix A for the logical flow of the project program.

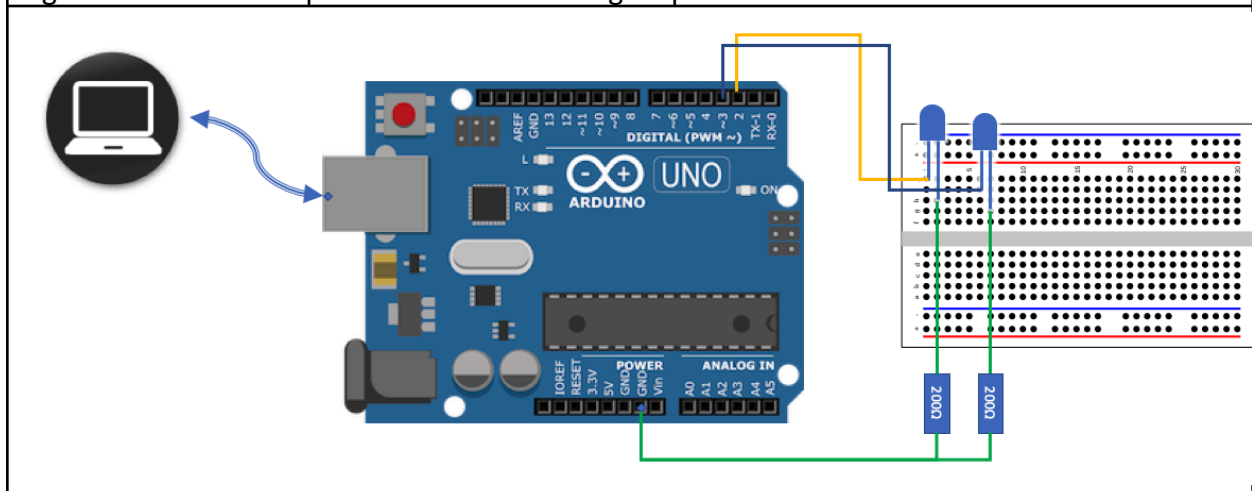
See Appendix B for the code executing on the Arduino Uno R3.

TESTING AND VALIDATION

<list requirements derived and any scenarios used. Including any testbed and other information need to perform the testing>

DESCRIPTION: The tests that have to be performed and validated are shown in Table xxx. These tests were performed on the testbed shown Figure 1. Table 1 shows a dialog that must be successfully performed by the embedded system. The results of testing are shown in Table 3. When necessary, a video of the test is provided along with this report.

Figure 1. Testbed Setup. LEDs connected to digital pins 2 & 3.



TESTING PLATFORM: *<include any equipment, SW, etc necessary to reproduce your design environment>*

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better
3. Power: Testing platform powered through USB cable, LEDs connected directly through Digital Outputs

Table 1. Required Test Dialog (*blue* are the user inputs).

Serial port I/O	Notes
What LED? (1 or 2) <i>2</i>	
What interval (in msec)? <i>600</i>	LED2 starts blinking at an interval of 300ms on and 300ms off. LED1 is unaffected.
What LED? (1 or 2) <i>1</i>	
What interval (in msec)? <i>1600</i>	LED1 starts blinking at an interval of 800ms on and 800ms off. LED2 is unaffected.
What LED? (1 or 2)	Waiting for next LED# interval pair

Button Press (6 times individually)	Fan turns on and changes speed per button press, before turning off on the 6th one.
-------------------------------------	---

TESTING AND VALIDATION REQUIREMENTS: *<include all tests using appropriate language, e.g. must, so that there is no ambiguity>*

Table 2. Testing and Validation Requirements

T.0 All testing and validation must be done on the testbed illustrated in Figure 1.
T.1 The dialog above (or similar) must work as shown
T.1A The blink rate of an LED must be able to be set without interfering with the blinking of the other LED
T.1.1 Setting of the blink rate (and LED#) must be through the IDE serial monitor
T.2 The blink rate of the LED being set must not change until the input is complete
T.3 The user must specify the blink rate in milliseconds per blink
T.4 The blink rate specified by the user must be correctly reflected on the testbed LEDs.
T.4.1 The blink rate specified by the user must be reflected on the LED specified by the user
T.5. The setting of an LED's blink rate must be able to be repeated at least 5 times
T.5.1. Successively for the same LED
T.5.2. Alternately between the different LEDs
T.6 The Fan must spin at various speeds

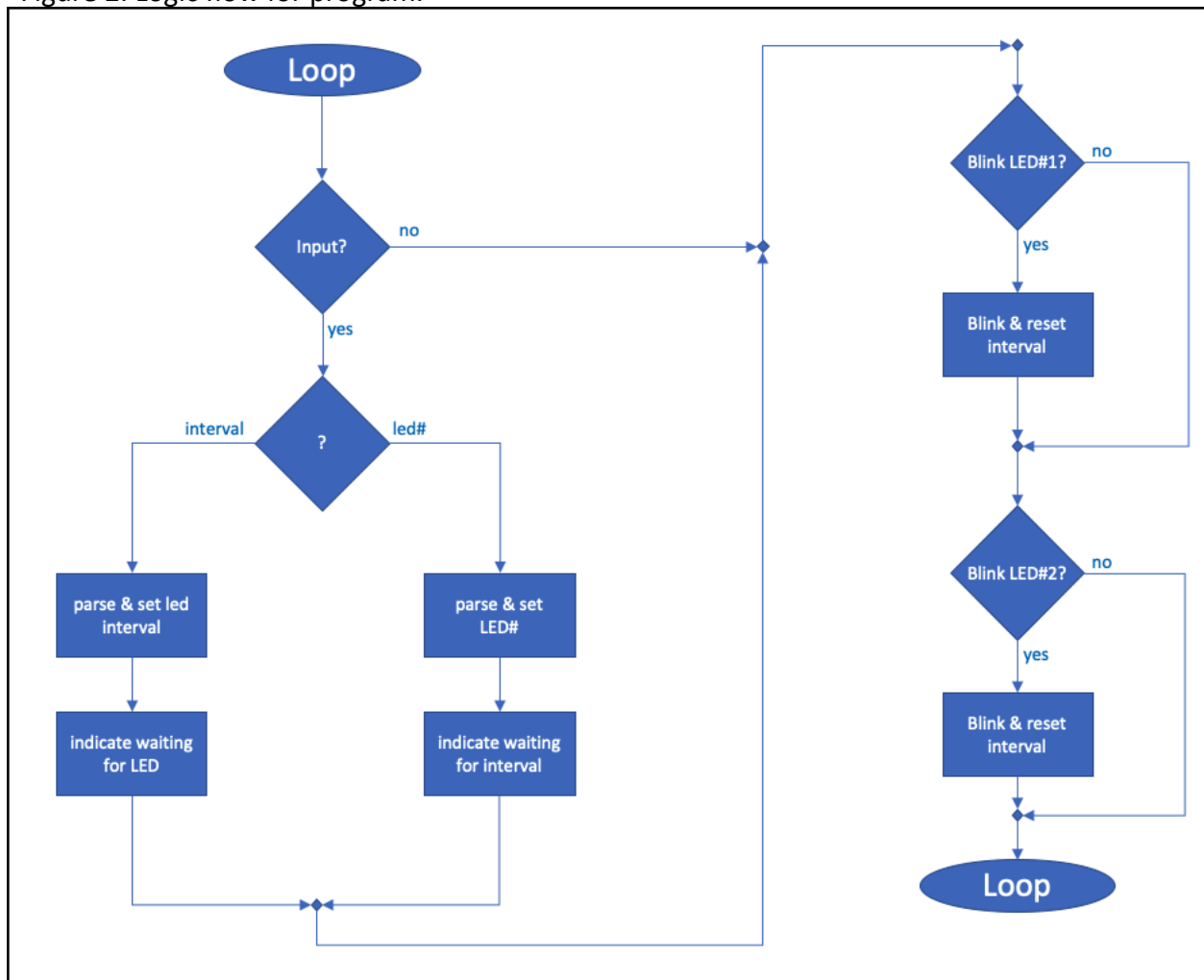
TESTING RESULTS: *<include any relevant tests, data, videos, etc that are used to confirm the tests>*

Table 3. Results of tests

Test performed	Results
T.1.1	Satisfied. See video of test.
T.1	Satisfied
T.2	Satisfied. See video of test.
T.3	Satisfied
T.4	Satisfied
T.4.1	Satisfied. See video of test.
T.5.1	Satisfied. See video of test.
T.5.2	Satisfied. See video of test.
T.6	Satisfied. Video may show inconclusive, check in lab. (Two people needed to test due to poor wire connections)

Appendix A. Logic Flow Chart

Figure 2. Logic flow for program.



APPENDIX B. DESIGN CODE

```
#include <Arduino_FreeRTOS.h>
#include <EEPROM.h>
#include <semphr.h>

// constants won't change. Used here to
// set pin numbers:
const int ledPin1 = 13; // the number of the LED pin
const int ledPin2 = 11;
const int inputPin = 2;
const int outputPin = 3;

int fanstate = 0;
int fanspeed = 0;

bool buttonCooldown = false;

// Variables will change:
int ledState1 = LOW; // ledState used to set the LED
int ledState2 = LOW;
int previousMillis1 = 0; // will store last time LED was updated
int previousMillis2 = 0;

// the follow variables is a long because the time, measured in milliseconds,
// will quickly become a bigger number than can be stored in an int.
int interval1 = 1000; // interval at which to blink (milliseconds)
int interval2 = 700;
bool userAsk = false;
bool userSpeed = false;
int ledSelect = -1;
int ledSpeed = 0;
```

```

bool skip = false;

unsigned int currentMillis1 = millis();
unsigned int currentMillis2 = millis();

SemaphoreHandle_t xSerialSemaphore;

int fn0() {
    Serial.print("\n");
    Serial.print("Select which LED you want to change\n");
    userAsk = true;
    delay(1);
    return 0;
}

int fn1() {
    ledSelect = Serial.parseInt();
    Serial.print(ledSelect);
    if (ledSelect == 1 || ledSelect == 2) {
        Serial.print(F("How fast do you want it to blink (ms)"));
        userSpeed = true;
    } else if (ledSelect != 10) {
        Serial.print(F("Invalid input, must be 1 or 2, input again\n"));
    }

    delay(1);
    return 0;
}

int fn2() {
    ledSpeed = Serial.parseInt();
    Serial.print(ledSpeed);
    if (ledSelect == 1 && ledSpeed > -1) {
        interval1 = ledSpeed;
        userSpeed = false;
        userAsk = false;
        EEPROM.put(0,interval1);
    }
}

```

```

    } else if (ledSelect == 2 && ledSpeed > -1) {
        interval2 = ledSpeed;
        userSpeed = false;
        userAsk = false;
        EEPROM.put(1,interval2);
    }

    delay(1);
    return 0;
}

int fn3() {
    // save the last time you blinked the LED
    previousMillis1 = currentMillis1;

    // if the LED is off turn it on and vice-versa:
    if (ledState1 == LOW)
        ledState1 = HIGH;
    else
        ledState1 = LOW;

    // set the LED with the ledState of the variable:
    digitalWrite(ledPin1, ledState1);
    delay(1);
    return 0;
}

int fn4() {
    // save the last time you blinked the LED
    previousMillis2 = currentMillis2;

    // if the LED is off turn it on and vice-versa:
    if (ledState2 == LOW)
        ledState2 = HIGH;
    else
        ledState2 = LOW;

    // set the LED with the ledState of the variable:

```

```
    digitalWrite(ledPin2, ledState2);
    delay(1);
    return 0;
}

int fn5(){
    if(fanstate==0){
        fanstate = 1;
        fanspeed = 1;
    }
    else if(fanstate==1){
        fanstate = 2;
        fanspeed = 2;
    }
    else if(fanstate == 2){
        fanstate = 3;
        fanspeed = 3;
    }
    else if(fanstate == 3){
        fanstate = 4;
        fanspeed = 2;
    }
    else if(fanstate == 4){
        fanstate = 5;
        fanspeed = 1;
    }
    else{
        fanstate = 0;
        fanspeed = 0;
    }

    buttonCooldown = true;
    analogWrite(outputPin, fanspeed*(255/3));
    delay(1);
    Serial.print(fanspeed);
    return 0;
}
```

```

int fn6(){
    int temp = 5/3;
    analogWrite(outputPin,temp*fanspeed);
    EEPROM.put(2,fanspeed);
    delay(1);
return 0;
}

int fn7(){
    buttonCooldown = false;
delay(1);
    return 0;
}

int (*fnptr[8])() = {fn0,fn1,fn2,fn3,fn4,fn5,fn6,fn7};

void setup() {
    Serial.begin(9600);
    Serial.setTimeout(5000);

    EEPROM.get(0, interval1);
    EEPROM.get(1,interval2);
    EEPROM.get(2,fanspeed);

    Serial.print("Interval 1: ");
    Serial.println(interval1);
    Serial.print("Interval 2: ");
    Serial.println(interval2);
    Serial.print("Fanspeed: ");
    Serial.println(fanspeed);

    if(interval1 < 0 || interval1 > 10000){interval1 = 1000;}
    if(interval2 < 0 || interval2 > 10000){interval2 = 700;}
    if(fanspeed < 0 || fanspeed > 5){fanspeed = 0;}
}

```



```

// set the digital pin as output:
pinMode(ledPin1, OUTPUT);
pinMode(ledPin2, OUTPUT);
currentMillis1 = millis();
currentMillis2 = millis();
if ( xSerialSemaphore == NULL ) // Check to confirm that the Serial
Semaphore has not already been created.
{
    xSerialSemaphore = xSemaphoreCreateMutex(); // Create a mutex semaphore we
will use to manage the Serial Port
    if ( ( xSerialSemaphore ) != NULL )
        xSemaphoreGive( ( xSerialSemaphore ) ); // Make the Serial Port
available for use, by "Giving" the Semaphore.
}
xTaskCreate(MyTask1, "Task2", 100, NULL, 2, NULL);

xTaskCreate(MyTask2, "Task2", 100, NULL, 2, NULL);
}

void loop() {

}

static void MyTask1(void* pvParameters)
{
    while(1)
    {
        if (!userAsk) {
            fnptr[0]();
        }

        if (Serial.available() > 0 && !userSpeed) {
            fnptr[1]();
        }
    }
}

```

```

    if (Serial.available() > 0 && userSpeed) {
        fnptr[2]();
    }

    if (currentMillis1 - previousMillis1 > interval1) {
        fnptr[3]();
        digitalWrite(ledPin2, HIGH);
    }

    if (currentMillis2 - previousMillis2 > interval2) {
        fnptr[4]();
    }
    currentMillis1 = millis();
    currentMillis2 = millis();
    if ( xSemaphoreTake( xSerialSemaphore, ( TickType_t ) 5 ) == pdTRUE )
    {
        xSemaphoreGive( xSerialSemaphore ); // Now free or "Give" the Serial Port
for others.
    }
    vTaskDelay(110/portTICK_PERIOD_MS);
}

}

static void MyTask2(void* pvParameters)
{

    while(1)

    {
        if(digitalRead(inputPin) == 1 && !buttonCooldown){
            Serial.print("Pressed Button\n");

```

```

    fnptr[5]();
    fnptr[6]();

}

if(digitalRead(inputPin) == 0 && buttonCooldown){
    Serial.print("Button unpressed");

    fnptr[7]();
    if ( xSemaphoreTake( xSerialSemaphore, ( TickType_t ) 5 ) == pdTRUE )
    {
        xSemaphoreGive( xSerialSemaphore ); // Now free or "Give" the Serial Port
for others.
    }

}

vTaskDelay(120/portTICK_PERIOD_MS);
}
}

```

END OF DOCUMENT